

LLM 및 TRANSFORMER 아키텍처

LSTM에서 Transformer까지 - LLM 추론 원리 체감 실습

실습 사전 학습 자료 - 모델 비교 실습부터 API 레벨 에이전트 구현까지

핵심 개념 : LSTM · Self-Attention · Tokenization · Embedding · CoT·SC·ReAct 기반 에이전트

2026 임성열 교수 Ph.D.

LSTM • Transformer • API



이 실습에서 배우는 세 가지

1

모델 한계 체감력

LSTM으로 직접 문장을 생성해보고, 장기 의존성 같은 한계를 눈으로 확인하는 연습입니다.

2

구조 개선 비교력

동일한 과제를 Transformer로 다시 풀어보며 Self-Attention이 무엇을 해결하는지 비교하는 연습입니다.

3

API 레벨 구현력

Transformer 기반 LLM을 에이전트 프레임워크로 호출해, 추론 과정을 응용 서비스 레벨에서 구현하는 연습입니다.

실습 진행 순서



팀 구성 : 실습1 개인 · 실습2 3인 1팀 · ※ 시간 배분은 예시이며 실습 방향에 맞게 조정 가능합니다.

실습에 필요한 핵심 개념

LSTM

장기 기억 순환망

이전 상태를 참조해 다음 단어를 예측하는 구조
· 문장이 길어질수록 앞부분 정보를 잊는 장기 의존성 한계가 있음

SA

Self-Attention

문장 내 모든 토큰이 서로를 한 번에 참조해 관계를 계산하는 Transformer의 핵심 연산

T/E

Tokenization · Embedding

입력 문장을 토큰 단위로 쪼개고, 각 토큰을 의미를 담은 벡터로 변환하는 전처리 과정

INF

Inference 파이프라인

토큰화 → 임베딩 → Self-Attention 연산을 거쳐 다음 토큰을 예측하는 과정을 반복하며 문장을 생성

P.E.

CoT · SC · ReAct

Transformer의 추론 위에서 동작하는 프롬프트 기법. 단계별 사고·교차검증·도구호출의 동작 원리를 복습

오늘 실습은 이 개념을
LSTM↔Transformer 비교와
API 레벨 에이전트 구현으로 직접 체감
합니다.

LSTM (Long Short-Term Memory)

게이트로 정보를 조절하며 순차적으로 다음 단어를 예측하는 순환신경망

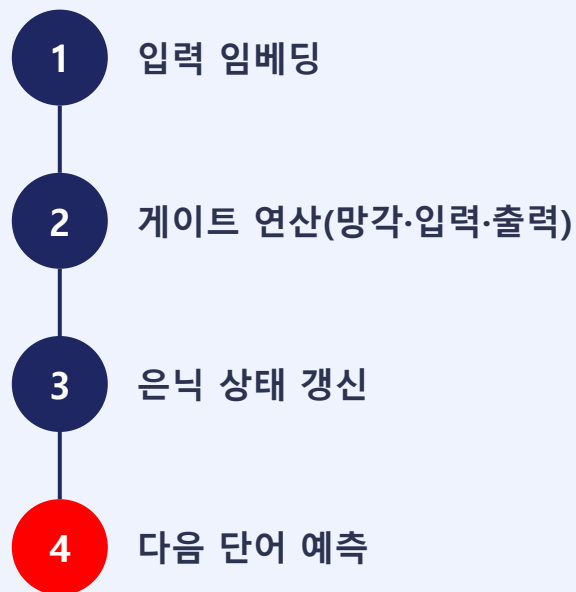
핵심 아이디어

이전 시점의 은닉 상태(hidden state)를 다음 시점으로 전달하며 순차적으로 다음 단어를 예측하는 순환신경망(RNN) 계열 구조입니다. 입력·출력·망각 게이트로 정보를 얼마나 유지하고 지울지 조절합니다.

왜 중요한가

게이트 구조 덕분에 단순 RNN보다 오래 정보를 유지할 수 있지만, 문장이 매우 길어지면 여전히 앞부분 정보가 희석되는 한계가 있습니다. 이 한계를 실습에서 직접 확인하는 것이 핵심입니다.

동작 흐름



LSTM (Long Short-Term Memory)

실습에서 왜 중요하고, 무엇을 조심해야 하는가?

핵심 강점

- 순서 정보를 구조적으로 자연스럽게 반영
- 구조가 상대적으로 단순해 적은 데이터·자원으로도 학습 가능
- 짧은 문장에서는 안정적인 생성 성능

한계·주의점

- 문장이 길어질수록 앞부분 정보가 희석되는 장기 의존성 문제
- 토큰을 하나씩 순차 처리해 병렬화가 어렵고 속도가 느림
- 이 한계가 바로 Transformer 등장의 배경이 됨

LSTM (Long Short-Term Memory)

실습에서 이 개념이 어떻게 나타나는가 · 적용 대상 : 실습 1

실습1에서 LSTM으로 긴 문장을 생성해보면 이 한계가 눈으로 드러납니다.

관찰 포인트

실습 1

예 : "오늘 아침 회사에서 팀 회의를 했는데..."로 시작한 문장이
뒤로 갈수록 회의 맥락과 무관한 내용으로 흘러가는 현상이 관찰됩니다.
이는 은닉 상태에 누적된 초반 정보가 뒤로 갈수록 희석되기 때문입니다.
hidden size나 문장 길이를 바꿔가며 이 현상이 언제부터 심해지는지
직접 비교해보는 것을 권장합니다.

LSTM (Long Short-Term Memory)

실습 채점 시 이 개념의 이해를 확인하는 방법

- LSTM의 게이트 구조(입력·출력·망각)를 설명할 수 있는가?
- 장기 의존성 문제가 실제 생성 결과에서 어떻게 나타나는지 예시로 설명할 수 있는가?
- 이 한계가 왜 Transformer의 필요성으로 이어지는지 연결해 설명할 수 있는가?

이 개념은 실습 1 실습에서 직접 확인할 수 있습니다.

Self-Attention

모든 토큰 간 관계를 한 번에 계산하는 Transformer의 핵심 연산

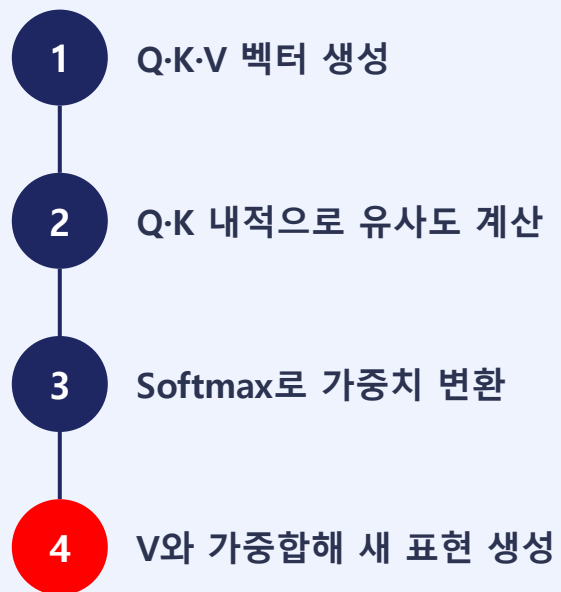
핵심 아이디어

문장 내 모든 토큰 쌍의 관계를 한 번의 행렬 연산으로 동시에 계산하는 연산입니다. Query-Key-Value 세 벡터의 유사도로 토큰 간 연관도 (attention score)를 구하고, 이를 가중치로 각 토큰의 표현을 갱신합니다.

왜 중요한가?

모든 토큰이 순서와 무관하게 서로를 직접 참조할 수 있어, LSTM이 겪던 장기 의존성 문제를 구조적으로 해결합니다. Transformer 아키텍처의 핵심 연산입니다.

동작 흐름



Self-Attention

실습에서 왜 중요하고, 무엇을 조심해야 하는가?

핵심 강점

- 거리와 무관하게 모든 토큰 간 관계를 직접 포착
- 토큰별 연산을 동시에 처리해 병렬화·학습 속도에서 유리
- LSTM 대비 장기 의존성 문제를 구조적으로 해결

비용·주의점

- 문장 길이의 제곱(N^2)에 비례해 연산량이 증가 - 긴 문서일수록 비용 부담
- 토큰 순서를 자체적으로 담지 못해 별도의 Positional Encoding이 필요
- 연산량 문제로 컨텍스트 윈도우(입력 가능 길이)에 현실적 한계가 존재

Self-Attention

실습에서 이 개념이 어떻게 나타나는가 · 적용 대상 : 실습 1·2

실습1·2 모두에서 Self-Attention의 효과를 직접 확인할 수 있습니다.

관찰 포인트

실습 1·2

실습1 : 동일 문장을 Transformer로 생성하면, LSTM에서 놓쳤던
앞부분 맥락(주어·인물관계 등)이 문장 후반까지 잘 유지됩니다.

실습2 : LLM API 호출 시 프롬프트 전체 토큰에 Self-Attention이
수행되어, 프롬프트 앞부분의 지시사항도 뒷부분 생성에 반영됩니다.

Self-Attention

실습 채점 시 이 개념의 이해를 확인하는 방법

- Query·Key·Value 각각의 역할을 설명할 수 있는가?
- LSTM 대비 장기 의존성 문제를 어떻게 해결하는지 설명할 수 있는가?
- N^2 연산량 트레이드오프를 언급할 수 있는가?

이 개념은 실습 1.2 실습에서 직접 확인할 수 있습니다.

Tokenization · Embedding

문장을 토큰으로 쪼개고 의미를 담은 벡터로 변환하는 전처리 과정

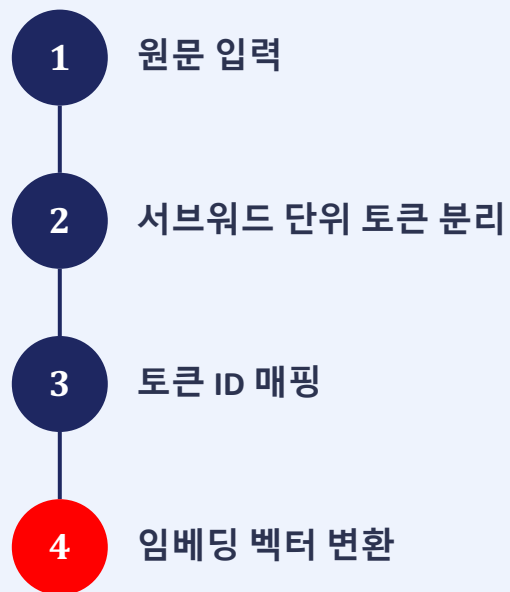
핵심 아이디어

입력 문장을 모델이 처리할 수 있는 단위(토큰)로 쪼개고, 각 토큰을 의미를 담은 고차원 벡터로 변환하는 전처리 과정입니다. 토큰화는 서브워드 단위로 문장을 분리하고, 임베딩은 그 토큰을 숫자 벡터로 매핑합니다.

왜 중요한가

신경망은 텍스트를 직접 처리할 수 없어 숫자로의 변환이 필수입니다. 서브워드 토큰화(BPE 등)는 신조어·오타자에도 유연하게 대응할 수 있습니다.

동작 흐름



Tokenization · Embedding

실습에서 왜 중요하고, 무엇을 조심해야 하는가?

왜 필요한가

- 텍스트를 신경망이 연산 가능한 숫자 벡터로 변환하는 필수 과정
- 서브워드 단위 토큰화로 신조어·오타자에도 유연하게 대응
- 의미가 비슷한 단어일수록 임베딩 공간에서 가까운 위치에 배치됨

주의할 점

- 토큰 수가 곧 API 비용·처리 속도에 직결됨(대부분 토큰 단위 과금)
- 언어별 토큰화 효율이 달라 같은 의미라도 언어마다 토큰 수가 다름
- 한국어는 영어보다 동일 의미 문장에 더 많은 토큰이 소요되는 경향

Tokenization · Embedding

실습에서 이 개념이 어떻게 나타나는가 · 적용 대상: 실습 2

실습2에서 API 응답의 토큰 사용량 (Usage)을 통해 직접 확인할 수 있습니다.

관찰 포인트

실습 2

동일한 의미의 프롬프트라도 한국어가 영어보다 토큰을 더 많이 소비하는 경우가 흔합니다. 이는 곧 비용과 응답 지연에 직결되므로, 에이전트를 설계할 때 프롬프트 길이·언어 선택을 함께 고려해야 합니다.

Tokenization · Embedding

실습 채점 시 이 개념의 이해를 확인하는 방법

- 토큰화와 임베딩의 차이를 설명할 수 있는가?
- 토큰 수가 비용·성능에 미치는 영향을 실습2 결과로 설명할 수 있는가?
- 서브워드 토큰화 방식의 장점을 설명할 수 있는가

이 개념은 실습 2 실습에서 직접 확인할 수 있습니다.

Inference 파이프라인

토큰화→임베딩→Self-Attention을 반복하며 한 토큰씩 문장을 생성하는 과정

핵심 아이디어

사용자 입력이 토큰화 → 임베딩 → Self-Attention 연산(여러 층 반복)을 거쳐 다음 토큰의 확률분포를 계산하고, 가장 적절한 토큰을 하나씩 생성하는 과정입니다. 생성된 토큰은 다시 다음 입력에 포함되어 반복됩니다(자기회귀 생성).

왜 중요한가

이 파이프라인을 이해하면 LLM API가 왜 토큰 단위로 스트리밍 응답을 줄 수 있는지, 왜 응답 속도가 출력 길이에 비례하는지 이해할 수 있습니다.

동작 흐름



Inference 파이프라인

실습에서 왜 중요하고, 무엇을 조심해야 하는가?

왜 알아야 하는가

- LLM API의 스트리밍 응답 방식(토큰 단위 출력)을 이해할 수 있음
- 응답 속도가 왜 출력 길이에 비례하는지 설명할 수 있음
- 에이전트가 LLM을 여러 번 호출할 때 비용이 왜 누적되는지 이해 가능

주의할 점

- 매 토큰 생성마다 이전 전체 문맥에 대한 연산이 반복됨(캐싱으로 일부 최적화)
- 컨텍스트 윈도우(입력 가능 길이) 제한이 곧 모델이 참조 가능한 범위의 한계
- 다단계 에이전트는 이 파이프라인을 여러 번 반복 호출해 속도·비용 부담이 커짐

Inference 파이프라인

실습에서 이 개념이 어떻게 나타나는가? · 적용 대상 : 실습 2

실습2의 에이전트 구조와 직접 연결됩니다.

관찰 포인트

실습 2

에이전트가 Thought → Action → Observation처럼 여러 단계를 거치며 LLM을 여러 번 호출한다면, 매 호출마다 이 추론 파이프라인 전체가 처음부터 다시 수행됩니다. 이것이 다단계 에이전트가 단일 호출보다 느리고 비용이 큰 이유입니다.

Inference 파이프라인

실습 채점 시 이 개념의 이해를 확인하는 방법

- 자기회귀 생성 과정 (한 토큰씩 예측→누적)을 설명할 수 있는가?
- 실습2 에이전트의 다단계 호출 구조와 추론 비용을 연결해 설명할 수 있는가?
- 컨텍스트 윈도우 제한이 실습 설계에 미치는 영향을 설명할 수 있는가?

이 개념은 실습 2 실습에서 직접 확인할 수 있습니다.

CoT · SC · ReAct

Transformer의 추론 위에서 동작하는 프롬프트 설계 기법 복습

핵심 아이디어

Transformer의 추론(다음 토큰 예측) 위에서 동작하는 프롬프트 설계 기법들입니다. 모델 구조를 바꾸지 않고 입력(프롬프트) 설계만으로 추론 품질을 끌어올립니다.

왜 중요한가?

실습2의 에이전트가 여러 단계로 작업을 수행할 때, 각 단계의 프롬프트에 이 기법들을 반영해야 안정적으로 동작합니다. Prompt & Context Engineering 수업에서 배운 내용과 직접 연결됩니다.

동작 흐름



CoT · SC · ReAct

실습에서 왜 중요하고, 무엇을 조심해야 하는가?

왜 필요한가

- 모델 구조를 바꾸지 않고 프롬프트만으로 추론 품질을 개선
- 에이전트의 각 단계(Task)를 안정적으로 동작시키는 설계 도구
- 이전 과목(Prompt & Context Engineering)과 개념적으로 직접 연결됨

주의할 점

- 모델 내부 구조(Self-Attention 등)를 바꾸는 것이 아니라 입력 설계일 뿐임
- 프롬프트를 아무리 잘 설계해도 컨텍스트 윈도우 같은 모델 자체의 한계는 극복 불가
- SC처럼 여러 경로를 도는 기법은 호출 횟수만큼 비용이 늘어남

CoT · SC · ReAct

실습에서 이 개념이 어떻게 나타나는가? · 적용 대상 : 실습 2

실습2 에이전트의 Task 설명에 ReAct 스타일을 반영한 예시입니다.

관찰 포인트

실습 2

Task: 고객 문의에 답하기 전 다음을 따르세요.

Thought: 먼저 사내 정책 문서에서 관련 조항을 조회해야 한다.

Action: 정책 문서 검색 도구를 호출하라.

Observation: [검색 결과]

Thought: 검색 결과를 바탕으로 최종 답변을 작성한다.

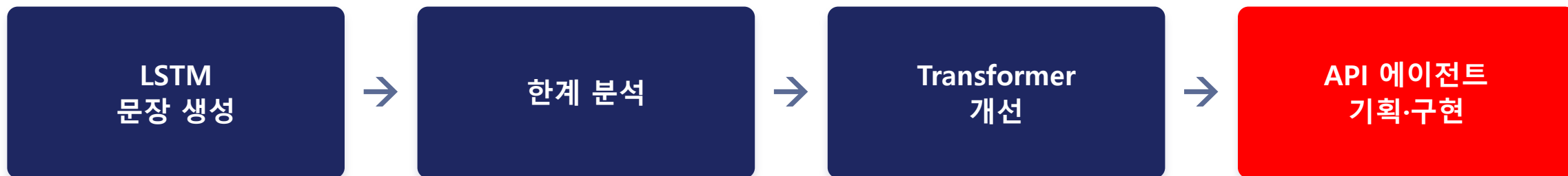
CoT · SC · ReAct

실습 채점 시 이 개념의 이해를 확인하는 방법

- 실습2 에이전트 프롬프트에 CoT-ReAct 구조가 실제로 반영되어 있는가?
- 프롬프트 기법과 모델 구조(Self-Attention)의 차이를 구분해 설명할 수 있는가?
- 이전 과목에서 배운 기법을 이번 실습에 연결해 설명할 수 있는가?

이 개념은 실습 2 실습에서 직접 확인할 수 있습니다.

실습 파이프라인 → 평가 연결



실습 1 (LSTM vs Transformer)과 실습 2 (API 에이전트)는 서로 다른 평가기준을 사용합니다.

실습 1 평가기준

정상 동작 여부 중심 - LSTM 문장 생성 40점 + Transformer 문장 생성 60점 (총 100점)

실습 2 평가기준

Biz가치·기술이해도·수업충실도 중심 - 40 + 30 + 30점 (총 100점)

1

딥러닝(LSTM) vs Transformer 모델 비교 실습평가

단계별 동작 원리 이해 중심 실습

실습 내용

LSTM으로 문장을 생성한 후 한계점을 파악하고, Transformer 모델로 문장 생성을 개선하는 실습

주요 활동

CoT·SC·ReAct 각 기법을 이해하고, LSTM과 Transformer의 단계별 동작 원리를 비교 분석

평가 포인트

LSTM 문장 생성 정상 동작 여부(40점) · Transformer 문장 생성 정상 동작 여부 (60점)

제출 산출물 예시

LSTM 기반 생성 코드·결과 / Transformer 기반 생성 코드·결과 / 두 모델의 한계·개선점을 비교한 리포트

2 Transformer 기반 LLM을 API 레벨에서 활용한 실습 (3인 과제)

추론 파이프라인 체감 & 서비스 기획 중심 실습

실습 내용

프레임워크 기반 LLM 에이전트 구성 및 연동 실습 (예 : CrewAI 등)

주요 활동

LLM 호출 시 내부적으로 토큰화 → 임베딩 → Self-Attention 연산을 거쳐 다음 토큰을 예측하는 Transformer의 추론 과정을 응용 레벨에서 체감

평가 포인트

시나리오 기획의 Biz 가치 (40점) · 기술 이해도 (30점) · 수업 충실도 (30점)

제출 산출물 예시

3인 팀 단위 에이전트 구현 코드 + 서비스 시나리오 기획서를 발표자료 (최소 3페이지 + Biz 가치 · 기술 이해도 · 수업 충실도 어필) 형태로 구성

평가 기준 · 실습 1 (총 100점)

1-1

딥러닝(RNN 또는 LSTM)을 이용한 문장 생성 여부

40점

RNN 또는 LSTM이 문장 생성에 올바르게 적용되었으며 정상 동작 하는가?

1-2

Transformer 모델을 이용한 대체 문장 생성 여부

60점

Transformer 모델이 문장 생성에 올바르게 적용되었으며 정상 동작 하는가?

※ 정상 동작 여부(코드 실행 결과)를 기준으로 채점하는 실습입니다.

평가 기준 · 실습 2 (총 100점)

2-1 · Biz 가치

Transformer 기반 LLM을 API 레벨 활용 시나리오 기획이 AI 서비스로서 Biz 가치가 있는가?

40점

2-2 · 기술 이해도

Transformer 기반 LLM을 API 레벨 활용에 대한 기술적 이해도를 갖추고 구현했는가?

30점

2-3 · 수업 충실도

Transformer와 Transformer 아키텍처 수업에서 배운 내용을 실습에 충실히 반영했는가?

30점

공통 필수 기재 항목 (실습 1·2 공통)

평가 결과에는 ① 개인별 점수, ② 점수에 대한 판단 근거, ③ 보완사항을 반드시 포함합니다.

BEFORE YOU START

실습 전 체크리스트



- 1 실습1(LSTM vs Transformer)·실습2(API 에이전트) 진행 순서·시간 배분 사전 공지
- 2 실습2는 3인 1팀 과제이므로 팀 편성을 미리 확정
- 3 LLM API 키, 에이전트 프레임워크(예 : CrewAI) 실행 환경 사전 점검
- 4 평가표에 개인별 점수·판단 근거·보완사항 기재란이 포함되어 있는지 확인
- 5 실습1(정상 동작 중심)·실습2(Biz가치·기술이해도·수업충실도) 배점 기준 사전 공지

From step-by-step recurrence to all-at-once attention - let the stakeholders feel the difference.